Cada dato tiene un lugar

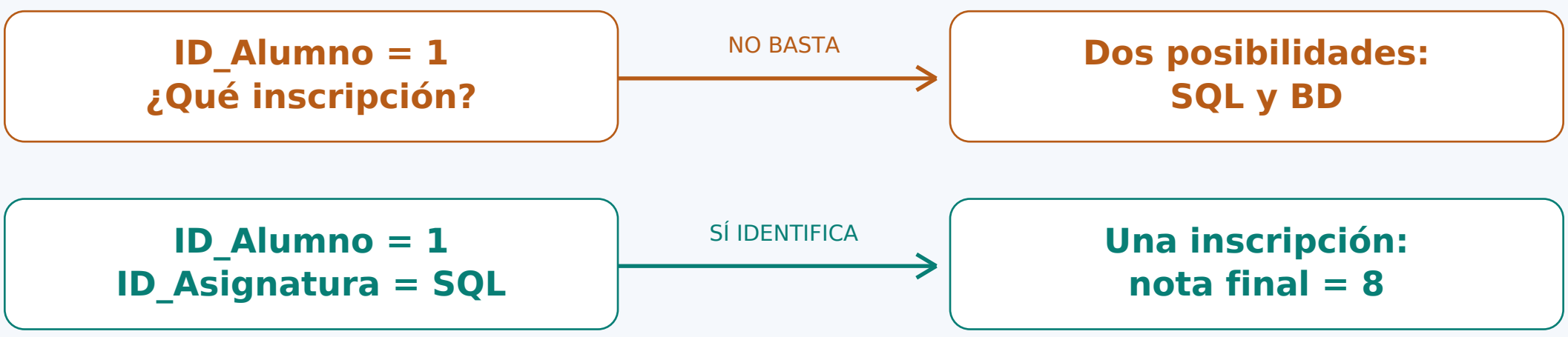
Normalizar es organizar los hechos para que cambiar uno no obligue a corregir muchas copias.

Nuestro pequeño mundo universitario

Una fila de Inscripciones representa un alumno en una asignatura. Suponemos una sola inscripción y una nota final por pareja. Si hubiera años académicos o reintentos, tendríamos que añadirlos al modelo.

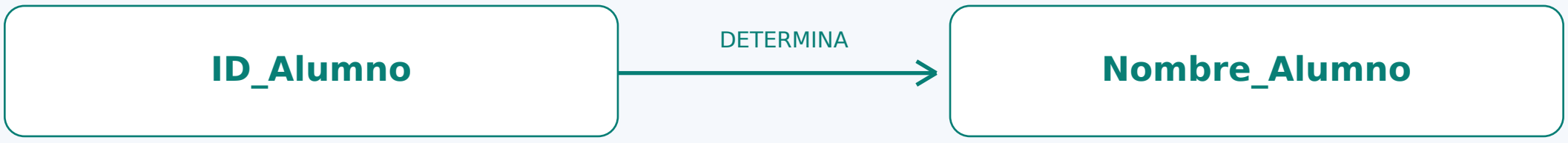
ID_Alumno	ID_Asignatura	Nombre_Alumno	Calificación
1	SQL	Ana	8
1	BD	Ana	9
2	SQL	Bruno	7

Una clave compuesta: dos datos identifican una fila



Clave candidata: conjunto mínimo de columnas que identifica una fila. Clave primaria (PK): la candidata que elegimos como identificador principal. Aquí es (ID_Alumno, ID_Asignatura).

¿Qué significa “depende de”?

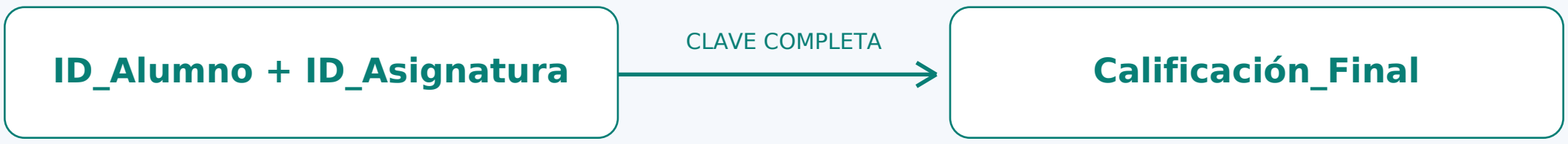


Si dos filas tienen el mismo ID_Alumno, deben indicar el mismo nombre actual. Es una regla del negocio, no una coincidencia de los datos que vemos hoy. No significa que el nombre no pueda cambiar ni que dos personas no puedan llamarse igual.

1NF, punto de partida: en este ejemplo cada celda contiene un valor del dominio, sin listas de inscripciones dentro de una celda. Ahora podemos estudiar de qué clave depende cada columna.

La mitad de la clave no basta

La pregunta: ¿este dato describe la inscripción completa o solo al alumno?



Para conocer la nota necesito saber quién y en qué asignatura. Ana tiene 8 en SQL y 9 en BD: su ID por sí solo no determina una nota.



Para conocer el nombre no necesito la asignatura. El nombre describe al alumno, aunque lo hayamos colocado en una tabla cuya fila representa una inscripción.

Lo que puede salir mal

Operación	Problema al guardar el nombre en cada inscripción
Actualizar	Ana cambia su nombre a Ana María. Si solo corrijo SQL, la fila de BD sigue diciendo Ana. ¿Cuál es el nombre actual?
Insertar	Quiero dar de alta a Carla antes de que elija asignaturas. Esta tabla exige una inscripción: no tiene un lugar adecuado para guardar solo a Carla.
Borrar	Si elimino la única inscripción de Bruno, también desaparece su nombre de esta tabla.

La regla de 2NF, en palabras sencillas

Partiendo de 1NF, cada atributo no clave debe depender de toda la clave compuesta, no solo de una parte. Para el caso sencillo de esta infografía, esa es la prueba que necesitamos.

No basta con preguntar “¿hay texto repetido?”. Dos alumnos distintos pueden tener el mismo nombre sin ningún problema. El problema es guardar muchas veces el mismo hecho: el nombre actual del mismo alumno.

Separa al alumno de su inscripción

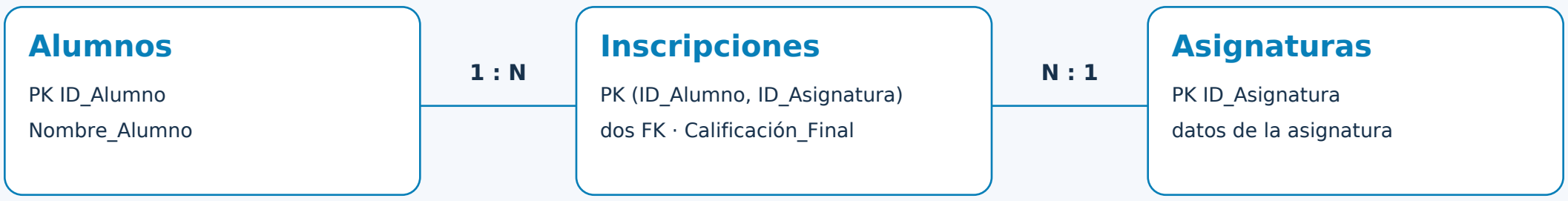
No perdemos información: la enlazamos mediante el ID del alumno.

ALUMNOS · Una fila por persona

ID_Alumno · PK	Nombre_Alumno
1	Ana
2	Bruno
3	Carla

INSCRIPCIONES · Una fila por alumno y asignatura

ID_Alumno · PK/FK	ID_Asignatura · PK/FK	Calificación_Final
1	SQL	8
1	BD	9
2	SQL	7



Las columnas con FK referencian los identificadores de Alumnos y Asignaturas. La tabla Asignaturas es el catálogo al que apuntan SQL y BD. Carla puede existir sin ninguna inscripción.

¿Y si quiero verlo todo junto?

```
SELECT a.ID_Alumno, a.Nombre_Alumno,
       i.ID_Asignatura, i.Calificacion_Final
FROM Alumnos AS a
JOIN Inscripciones AS i ON i.ID_Alumno = a.ID_Alumno;
```

El resultado vuelve a mostrar Ana dos veces. Eso está bien: la repetición se calcula al leer, no son dos copias independientes del nombre que haya que mantener al escribir.

Un cambio, una fila: actualizar Alumnos basta para que las consultas vean el nombre actual.

Crear un ID_Inscripcion artificial no arregla por sí solo el diseño anterior: la pareja alumno-asignatura sigue siendo una clave candidata si debe ser única. Hay que estudiar todas las candidatas.

Puede cumplir 2NF y fallar 3NF

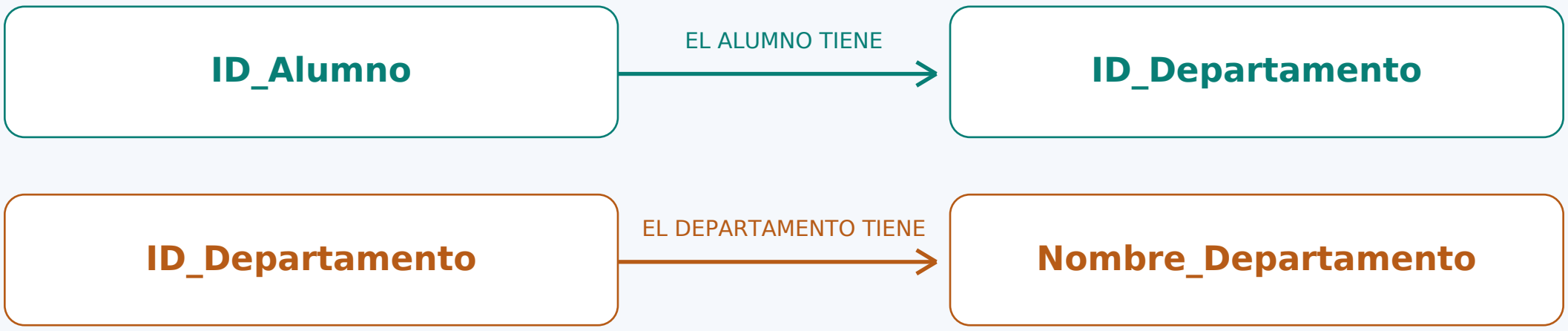
Añadimos al alumno su departamento y el nombre actual de ese departamento.

Reglas: cada alumno pertenece a un departamento; un departamento tiene un único nombre actual; varios alumnos pueden pertenecer al mismo departamento. Solo ID_Alumno identifica a un alumno en esta tabla.

ID_Alumno · PK	Nombre_Alumno	ID_Departamento	Nombre_Departamento
1	Ana	D10	Ingeniería
2	Bruno	D10	Ingeniería
3	Carla	D20	Historia

La clave tiene una columna: no hay “media clave”

En estas condiciones cumple 2NF: no puede haber una dependencia respecto de una parte de ID_Alumno. Pero Ingeniería sigue repetida para cada alumno de D10.



Una dependencia transitiva: llegamos a través de otro dato

ID_Alumno → ID_Departamento → Nombre_Departamento. El nombre del departamento no describe al alumno: describe la entidad departamento, identificada por otra columna no clave de esta tabla.

Cambio	Riesgo
D10 se renombra “Ingeniería y Tecnología”	Hay que corregir a todos sus alumnos; una fila olvidada deja nombres contradictorios.
Creamos D30 sin alumnos	No hay una fila adecuada donde registrar el nuevo departamento.
Se elimina el último alumno de D20	Podemos perder el único registro del nombre Historia.

La 3NF da el siguiente paso: en estos ejemplos, separa los datos que dependen de otro atributo que no es clave. Así evitamos almacenar hechos de otra entidad dentro de cada alumno.

Cada hecho con su propietario

El alumno guarda a qué departamento pertenece; el departamento guarda cómo se llama.

ALUMNOS · Conservamos la referencia

ID_Alumno · PK	Nombre_Alumno	ID_Departamento · FK
1	Ana	D10
2	Bruno	D10
3	Carla	D20

DEPARTAMENTOS · Un único nombre actual por ID

ID_Departamento · PK	Nombre_Departamento
D10	Ingeniería
D20	Historia
D30	Física

Relación 1:N: un departamento puede tener muchos alumnos.

D30 puede existir aunque no tenga alumnos. Cambiar el nombre de D10 requiere actualizar una fila. Inscripciones, la tabla que ya resolvimos en 2NF, sigue enlazada con Alumnos.

```
SELECT a.Nombre_Alumno, d.Nombre_Departamento
FROM Alumnos AS a
JOIN Departamentos AS d
  ON d.ID_Departamento = a.ID_Departamento;
```

Comparación que conviene recordar

2NF · dependencia parcial	3NF · dependencia transitiva
Clave (alumno, asignatura) → nombre del alumno. Solo alumno basta.	Clave alumno → departamento → nombre del departamento. Hay un intermediario no clave.
Extrae los datos que describen solo una parte de la clave.	Extrae los datos que describen otra entidad identificada por un atributo no clave.

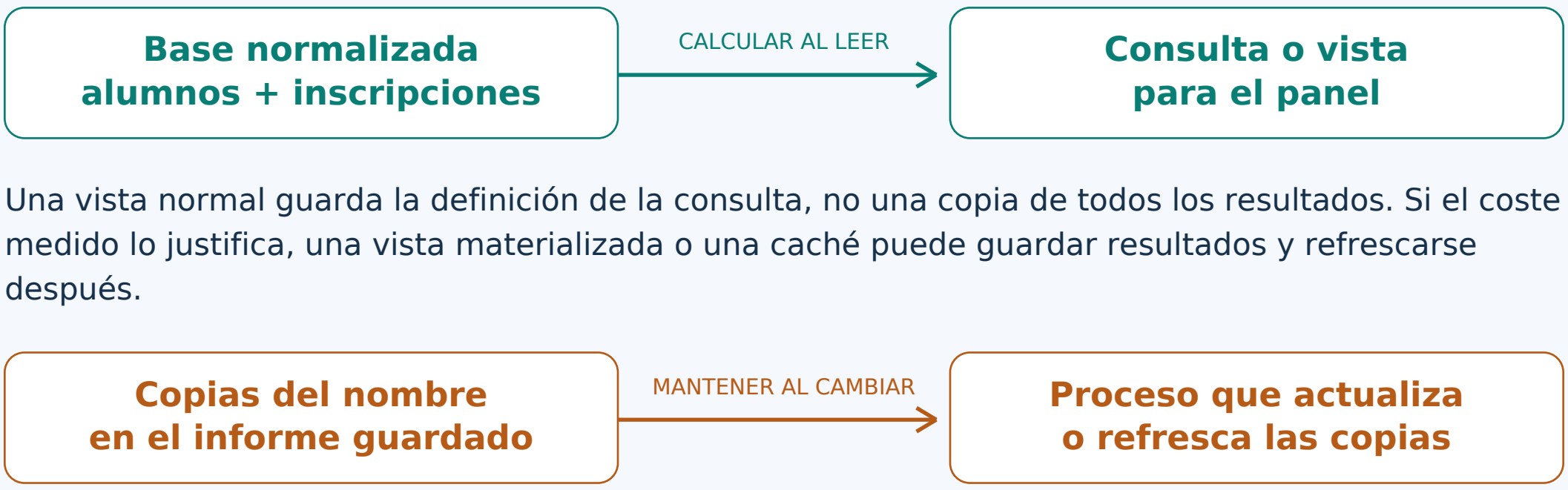
Guardar el mismo ID_Departamento en varios alumnos es correcto: registra varias pertenencias diferentes. Normalizar no significa prohibir valores repetidos ni eliminar las FK.

Normalizar también tiene un precio

La decisión práctica combina integridad, frecuencia de cambios y patrón de lectura. Menos JOIN no significa automáticamente más velocidad.

Diseño	Qué facilita	Qué exige o complica
2NF y 3NF	Un lugar para cada hecho. Menos copias contradictorias; altas, bajas y cambios independientes.	Más tablas, FK y consultas con JOIN; índices y transacciones bien diseñados.
Duplicación deliberada	Puede simplificar y acelerar una lectura frecuente al tener el informe ya reunido.	Más copias que actualizar, almacenamiento adicional y una estrategia explícita de coherencia.

Ejemplo: panel de 100.000 inscripciones



Una vista normal guarda la definición de la consulta, no una copia de todos los resultados. Si el coste medido lo justifica, una vista materializada o una caché puede guardar resultados y refrescarse después.

Debes decidir cuál es la fuente de verdad, quién propaga los cambios y cuánto retraso es tolerable. Una copia deliberadamente antigua puede servir para un panel; para otra operación podría ser inaceptable.

No toda diferencia es una inconsistencia

Nombre_Actual y Nombre_En_El_Diploma representan hechos distintos. Un diploma histórico puede conservar el nombre de su fecha aunque el alumno cambie de nombre hoy. No lo “corrijas” automáticamente: primero define qué significa cada dato.

Regla práctica

Empieza separando los hechos que cambian de forma independiente. Mide las consultas reales. Duplica solo si aporta una ventaja concreta y puedes explicar cómo mantendrás esa copia o por qué debe ser histórica.

Flexible no significa incoherente

Tu intuición tiene una parte correcta: MongoDB permite modelos embebidos cómodos para leer juntos los datos relacionados. Pero no obliga a duplicarlos.

Dos decisiones distintas

Flexibilidad del esquema	Normalización / duplicación
¿Pueden algunos alumnos tener campos adicionales, por ejemplo telefono o beca?	¿Guardo el nombre de D10 una vez o dentro de cada alumno?
MongoDB permite estructuras variables y también reglas de validación.	MongoDB admite referencias entre documentos y datos embebidos.

A · Referenciar: el nombre del departamento, una vez

```
// Colección alumnos
{ _id: 1, nombre: "Ana", departamento_id: "D10" }

// Colección departamentos
{ _id: "D10", nombre: "Ingeniería" }
```

Si necesitas ambos datos, puedes usar $lookup o consultas adicionales. Las referencias no son FK relacionales que se verifiquen automáticamente: debes cuidar esa integridad en el diseño y la lógica de la aplicación.

B · Embeber: todo lo necesario en el documento

```
// El departamento queda copiado en cada alumno.
{ _id: 1, nombre: "Ana",
  departamento: { id: "D10", nombre: "Ingeniería" } }
{ _id: 2, nombre: "Bruno",
  departamento: { id: "D10", nombre: "Ingeniería" } }
```

Una lectura obtiene alumno y departamento. A cambio, renombrar D10 exige mantener varias copias. La atomicidad de una escritura en un documento no sincroniza por sí sola los demás.

Embeber no siempre duplica hechos

Un alumno con un array de contactos propios puede guardar cada contacto una sola vez. Eso es anidación, pero no implica necesariamente repetir el mismo hecho en varios documentos. Conviene acotar el crecimiento del array.

2NF y 3NF son reglas del modelo relacional: no se aplican mecánicamente a un BSON anidado. La pregunta útil también aquí es: ¿dónde vive cada hecho y qué pasa cuando cambia?

08 · COMPRUÉBALO TÚ

Tres preguntas antes de separar

Las dependencias se deducen de las reglas del negocio. No basta con mirar tres filas y buscar valores repetidos.

Pregunta	Ejemplo	Diagnóstico
¿Una parte de la clave basta para conocer este dato?	(Alumno, Asignatura) → Nombre_Alumno; Alumno basta.	Problema de 2NF.
¿Un atributo no clave determina un dato de otra entidad?	Alumno → Departamento → Nombre_Departamento.	Problema de 3NF en este caso.
¿Solo repito información al mostrarla con un JOIN?	Ana aparece en SQL y BD, pero su nombre se guarda una vez.	No desnormaliza las tablas de origen.

Mini ejercicio: decide dónde guardar cada cosa

1. Nombre del alumno.
2. Nota del alumno en una asignatura.
3. Nombre del departamento.
4. Departamento al que pertenece el alumno.

Respuesta	Porque…
1 → Alumnos	Depende del identificador del alumno.
2 → Inscripciones	Depende de la pareja alumno-asignatura.
3 → Departamentos	Depende del identificador del departamento.
4 → Alumnos, como FK	Describe la pertenencia de ese alumno.

Matiz técnico, cuando quieras ir un paso más allá

2NF: estar en 1NF y no tener atributos no primos que dependan de una parte propia de ninguna clave candidata. “Primo” significa que pertenece a alguna clave candidata.

3NF: para cada dependencia funcional no trivial X → A, X es superclave o A es un atributo primo. Una superclave identifica filas, aunque pueda contener columnas de más. En nuestros ejemplos, esto se traduce en eliminar la dependencia transitiva problemática.

Por eso una PK de una columna no garantiza por sí sola toda la normalización: puede haber otras claves candidatas compuestas y, aunque se cumpla 2NF, aún puede fallar 3NF.

Fuentes y alcance

Ejemplos y diagramas propios. Referencias para ampliar:
Microsoft · Database design basics
https://support.microsoft.com/en-us/access/database-design-basics
MongoDB · Data Modeling
https://www.mongodb.com/docs/manual/data-modeling/
MongoDB · Data Consistency
https://www.mongodb.com/docs/manual/data-modeling/data-consistency/
MongoDB · Enforce Data Consistency with Embedding
https://www.mongodb.com/docs/manual/data-modeling/enforce-consistency/embed-data/